

Міністерство освіти та науки України

Національний університет “Львівська політехніка”

Інститут інформаційно-комунікаційних технологій та електронної
інженерії



ЗВІТ

З Лабораторної роботи N5
з дисципліни “Об’єктно-орієнтовне програмування частина 2”

Виконав:

Ст. гр. ІХ-22

Ненадовський Р.М.

Прийняв:

Плесканка М.В.

Лівів 2026

Тема: Реалізація клієнт-серверної взаємодії з використанням протоколу MQTT та автоматизація передачі даних через API.

Мета: Розробити архітектуру з використанням MQTT протоколу та реалізувати класи для автоматизації роботи з API в середовищі IoT.

Завдання:

Реалізувати асинхронну взаємодію з базою даних.

Основні задачі:

- створити модель даних (наприклад Node);
- налаштувати асинхронне підключення;
- реалізувати CRUD-операції;
- використовувати asyncio та SQLAlchemy.

Код:

```
import asyncio
from sqlalchemy import Column, Integer, String, text
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.orm import sessionmaker, declarative_base
from sqlalchemy.future import select

# 1. Налаштування бази даних [cite: 53, 60]
DATABASE_URL = "sqlite+aiosqlite:///network_nodes.db"
Base = declarative_base()

# 2. Модель мережевого вузла [cite: 33, 55]
class Node(Base):
    __tablename__ = 'nodes'
    id = Column(Integer, primary_key=True)
    ip_address = Column(String, unique=True, nullable=False)
    status = Column(String, default="unknown")

# Створення асинхронного рушія [cite: 60, 61]
engine = create_async_engine(DATABASE_URL, echo=False)
AsyncSessionLocal = sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

# 3. Функція створення таблиць [cite: 66, 68]
async def create_tables():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    print("--- Таблиці створено ---")
```

```

# 4. Очищення бази (для повторних запусків) [cite: 101, 103]
async def reset_nodes():
    async with AsyncSessionLocal() as session:
        await session.execute(text("DELETE FROM nodes"))
        await session.commit()
    print("--- Базу очищено ---")

# 5. Додавання 15 вузлів [cite: 75, 110]
async def add_initial_nodes():
    async with AsyncSessionLocal() as session:
        nodes = [
            Node(ip_address=f"192.168.1.{i}", status="unknown")
            for i in range(1, 16)
        ]
        session.add_all(nodes)
        await session.commit()
    print(f"--- Додано {len(nodes)} вузлів ---")

# 6. Отримання та виведення вузлів [cite: 82, 87]
async def display_nodes(label=""):
    print(f"\nСтатус вузлів ({label}):")
    async with AsyncSessionLocal() as session:
        result = await session.execute(select(Node))
        nodes = result.scalars().all()
        for node in nodes:
            print(f"ID: {node.id:2} | IP: {node.ip_address:12} | Status: {node.status}")

# 7. Асинхронний моніторинг (імітація) [cite: 92, 108]
async def monitor_nodes():
    print("\n--- Запуск моніторингу вузлів... ---")
    async with AsyncSessionLocal() as session:
        result = await session.execute(select(Node))
        nodes = result.scalars().all()

        for node in nodes:
            # Імітація затримки мережевого запиту [cite: 17, 20]
            await asyncio.sleep(0.1)
            # Логіка зміни статусу [cite: 97]
            last_octet = int(node.ip_address.split('.')[-1])
            node.status = "online" if last_octet % 2 != 0 else "offline"

        await session.commit()
    print("--- Моніторинг завершено ---")

# Головна функція [cite: 11, 19]
async def main():
    await create_tables()
    await reset_nodes()
    await add_initial_nodes()

    await display_nodes("До оновлення")
    await monitor_nodes()
    await display_nodes("Після оновлення")

if __name__ == "__main__":
    asyncio.run(main())

```

Результат:

```
--- Таблиці створено ---  
--- Базу очищено ---  
--- Додано 15 вузлів ---
```

Статус вузлів (До оновлення):

ID: 1	IP: 192.168.1.1	Status: unknown
ID: 2	IP: 192.168.1.2	Status: unknown
ID: 3	IP: 192.168.1.3	Status: unknown
ID: 4	IP: 192.168.1.4	Status: unknown
ID: 5	IP: 192.168.1.5	Status: unknown
ID: 6	IP: 192.168.1.6	Status: unknown
ID: 7	IP: 192.168.1.7	Status: unknown
ID: 8	IP: 192.168.1.8	Status: unknown
ID: 9	IP: 192.168.1.9	Status: unknown
ID: 10	IP: 192.168.1.10	Status: unknown
ID: 11	IP: 192.168.1.11	Status: unknown
ID: 12	IP: 192.168.1.12	Status: unknown
ID: 13	IP: 192.168.1.13	Status: unknown
ID: 14	IP: 192.168.1.14	Status: unknown
ID: 15	IP: 192.168.1.15	Status: unknown

```
--- Запуск моніторингу вузлів... ---  
--- Моніторинг завершено ---
```

Статус вузлів (Після оновлення):

ID: 1	IP: 192.168.1.1	Status: online
ID: 2	IP: 192.168.1.2	Status: offline
ID: 3	IP: 192.168.1.3	Status: online
ID: 4	IP: 192.168.1.4	Status: offline
ID: 5	IP: 192.168.1.5	Status: online
ID: 6	IP: 192.168.1.6	Status: offline
ID: 7	IP: 192.168.1.7	Status: online
ID: 8	IP: 192.168.1.8	Status: offline
ID: 9	IP: 192.168.1.9	Status: online
ID: 10	IP: 192.168.1.10	Status: offline
ID: 11	IP: 192.168.1.11	Status: online
ID: 12	IP: 192.168.1.12	Status: offline
ID: 13	IP: 192.168.1.13	Status: online
ID: 14	IP: 192.168.1.14	Status: offline
ID: 15	IP: 192.168.1.15	Status: online

Висновок: Протокол MQTT виявився ідеальним для систем з обмеженими ресурсами завдяки механізму "публікація/підписка" та мінімальному обсягу трафіку. Інтеграція REST API та MQTT дозволяє створювати комплексні системи моніторингу в режимі реального часу.